



BOLSA DE EMPLEO UNIPAZ

Documentación Técnica del Proyecto

Institución	Universidad de Paz — UNIPAZ, Barrancabermeja
Versión	1.0
Fecha	Abril 2026
Stack tecnológico	Laravel 12 · PHP 8.2 · SQLite · Blade · Bootstrap 5

Esta documentación explica de manera completa y paso a paso la arquitectura, componentes, base de datos, autenticación, módulos y despliegue del sistema Bolsa de Empleo UNIPAZ. Está orientada a estudiantes que se inician en el desarrollo web con Laravel y PHP.

Tabla de Contenido

Tabla de Contenido	2
1. Introducción y Descripción del Proyecto.....	4
Características principales del sistema.....	4
2. Stack Tecnológico	5
Backend.....	5
Base de Datos	5
Frontend	5
Herramientas de Desarrollo	5
3. Arquitectura MVC con Laravel.....	6
¿Qué es el patrón MVC?	6
Flujo de una petición HTTP en Laravel.....	6
Ejemplo concreto: el estudiante ve la lista de vacantes	6
4. Estructura de Carpetas del Proyecto.....	7
Organización de Controladores por Módulo	7
5. Base de Datos — Modelos y Relaciones	8
5.1 Tabla: users	8
5.2 Tabla: student_profiles.....	8
5.3 Tabla: companies.....	8
5.4 Tabla: job_postings.....	10
5.5 Tabla: applications.....	10
5.6 Relaciones entre tablas	10
6. Sistema de Autenticación	12
6.1 Autenticación de Estudiantes — Google OAuth 2.0	12
6.2 Autenticación de Empresas y Admin — Email y Contraseña.....	12
6.3 Registro de Empresas	12
6.4 Middleware de Roles — RoleMiddleware	13
7. Módulo de Administración	14
Funcionalidades del panel de administración	14
¿Cómo aprueba el admin una empresa?.....	14
8. Módulo de Empresas.....	15
Funcionalidades del panel de empresa	15
CRUD de Vacantes — JobPostingController.....	15
Validación al crear una vacante (extracto).....	15
Estados de las postulaciones	15
9. Módulo de Estudiantes	17
Funcionalidades del panel de estudiante.....	17
¿Cómo se postula un estudiante?	17
Query Scopes en el modelo JobPosting	17
10. Sistema de Notificaciones	19
¿Cómo funcionan las notificaciones en Laravel?.....	19
Estructura de una Notificación	19
11. Rutas y Middleware	20

Grupos de rutas y sus protecciones	20
Named Routes (Rutas con nombre)	20
Middleware CheckRole — Cómo funciona	20
12. Vistas — Blade Templates	21
Sintaxis básica de Blade	21
Organización de las vistas	21
Ejemplo de vista Blade — Lista de vacantes:	21
13. Instalación Local — Paso a Paso	22
Paso 1: Descomprimir el proyecto	22
Paso 2: Instalar dependencias PHP	22
Paso 3: Configurar el archivo .env	22
Paso 4: Ejecutar migraciones y datos de prueba	22
Paso 5: Configurar el almacenamiento	22
Paso 6: Iniciar el servidor de desarrollo	22
Credenciales de acceso para pruebas	22
Configurar Google OAuth (para login de estudiantes)	22
Configurar envío de correos (para notificaciones)	22
Iniciar el worker de colas	23
14. Despliegue en Railway (Producción)	24
Paso 1: Subir el código a GitHub	24
Paso 2: Crear proyecto en Railway	24
Paso 3: Configurar variables de entorno	24
Paso 4: Generar el dominio	24
Paso 5: Actualizar Google Cloud Console	24
Actualizar el proyecto en producción	24
15. Glosario de Términos	25

1. Introducción y Descripción del Proyecto

La Bolsa de Empleo UNIPAZ es una aplicación web desarrollada para la Universidad de Paz (UNIPAZ) ubicada en Barrancabermeja, Colombia. Su propósito es conectar a los estudiantes y egresados de la institución con empresas de la región que ofrecen oportunidades laborales.

El sistema resuelve tres necesidades principales:

- **Estudiantes:** Pueden buscar vacantes activas, postularse con su hoja de vida y hacer seguimiento al estado de sus postulaciones, todo desde un panel personalizado.
- **Empresas:** Se registran en la plataforma, son verificadas por el administrador de UNIPAZ, y luego pueden publicar, editar y gestionar vacantes laborales.
- **Administradores:** UNIPAZ controla quién puede publicar vacantes, gestiona usuarios y accede a reportes estadísticos del sistema.

Características principales del sistema

Característica	Descripción
Login con Google OAuth	Los estudiantes ingresan exclusivamente con su correo @unipaz.edu.co
Aprobación de empresas	El administrador verifica cada empresa antes de que pueda publicar vacantes
Gestión de vacantes	CRUD completo: crear, editar, pausar y cerrar vacantes con múltiples atributos
Sistema de postulaciones	Los estudiantes se postulan una sola vez por vacante; las empresas gestionan el proceso
Notificaciones por email	Alertas automáticas en cada cambio de estado (aprobación, postulación, entrevista, etc.)
Panel de reportes	El administrador visualiza estadísticas del sistema: usuarios, vacantes, postulaciones
Roles y permisos	Middleware de roles que protege cada sección según el tipo de usuario
Diseño responsivo	Interfaz adaptada a dispositivos móviles y de escritorio

2. Stack Tecnológico

El proyecto utiliza un conjunto de tecnologías modernas y ampliamente usadas en el desarrollo web profesional. A continuación se explica cada una y por qué fue elegida.

Backend

Tecnología	Versión	Función en el proyecto
PHP	8.2+	Lenguaje de programación del servidor. Potente para aplicaciones web.
Laravel	12.x	Framework PHP. Provee estructura MVC, ORM, autenticación, rutas, etc.
Laravel Sanctum	4.x	Manejo seguro de sesiones de API y autenticación de tokens.
Laravel Socialite	5.x	Integración con Google OAuth para login de estudiantes.
Composer	2.x	Gestor de dependencias PHP. Similar a npm para JavaScript.

Base de Datos

Motor	Uso	Observación
SQLite	Desarrollo y Railway	Archivo .sqlite en el proyecto. Ideal para demos y pruebas rápidas.
MySQL	Producción local	Recomendado para entornos de producción con alta concurrencia.

Frontend

Tecnología	Función
Blade (Laravel)	Motor de plantillas HTML. Permite usar lógica PHP dentro del HTML de forma limpia.
Bootstrap 5	Framework CSS. Proporciona componentes visuales (botones, tablas, formularios, etc.)
Vite	Empaquetador de assets modernos (JS/CSS). Reemplaza a Webpack en proyectos recientes.
JavaScript ES6	Pequeñas interacciones del cliente (validaciones, AJAX de notificaciones).

Herramientas de Desarrollo

Herramienta	Propósito
Git + GitHub	Control de versiones y repositorio remoto del código fuente.
Railway.app	Plataforma de despliegue en la nube (hosting gratuito con GitHub).
PHPUnit	Framework de pruebas unitarias para PHP, integrado en Laravel.
Artisan CLI	Herramienta de línea de comandos de Laravel para generar código, migraciones, etc.

3. Arquitectura MVC con Laravel

Laravel implementa el patrón de arquitectura MVC (Modelo - Vista - Controlador), que separa la lógica del negocio, la interfaz de usuario y el control del flujo de la aplicación. Esto hace el código más organizado, mantenible y fácil de entender.

¿Qué es el patrón MVC?

Capa	Responsabilidad	En este proyecto
Modelo (M)	Representa los datos y la lógica de negocio. Interactúa directamente con la base de datos.	User, Company, JobPosting, Application, StudentProfile
Vista (V)	Es la interfaz que ve el usuario final. En Laravel se usan archivos Blade (.blade.php).	resources/views/ (admin, company, student, auth)
Controlador (C)	Recibe la petición HTTP del usuario, llama al modelo y devuelve la vista correcta.	HomeController, DashboardController, JobPostingController...

Flujo de una petición HTTP en Laravel

Cada vez que el navegador solicita una página, Laravel sigue estos pasos:

- 1. El navegador envía una petición HTTP (ej: GET /empresa/vacantes).
- 2. Laravel revisa routes/web.php y busca qué controlador debe manejarla.
- 3. El Middleware de autenticación y roles verifica que el usuario tenga permiso.
- 4. El Controlador recibe la petición y consulta el Modelo si necesita datos.
- 5. El Modelo consulta la base de datos mediante Eloquent ORM.
- 6. El Controlador pasa los datos a la Vista (archivo .blade.php).
- 7. La Vista genera el HTML final que el navegador muestra al usuario.

Ejemplo concreto: el estudiante ve la lista de vacantes

Ruta definida en routes/web.php:

```
Route::get('/estudiante/vacantes', [JobController::class, 'index'])
    ->middleware(['auth', 'role:student'])
    ->name('student.jobs');
```

Controlador Student/JobController.php:

```
public function index(Request $request)
{
    $query = JobPosting::active()->with('company');
    if ($request->filled('area')) {
        $query->byArea($request->area);
    }
    $jobPostings = $query->latest()->paginate(12);
    return view('student.jobs.index', compact('jobPostings'));
}
```

El controlador llama a JobPosting::active() que es un Query Scope definido en el modelo. Este scope filtra automáticamente las vacantes con status = "active" y deadline mayor a hoy. Los resultados se pagan de 12 en 12 y se envían a la vista student.jobs.index.

4. Estructura de Carpetas del Proyecto

Laravel organiza el código en carpetas con responsabilidades muy definidas. Entender esta estructura es fundamental para navegar el proyecto.

Carpeta / Archivo	Descripción
app/Models/	Los modelos Eloquent: User, Company, JobPosting, Application, StudentProfile.
app/Http/Controllers/	Los controladores organizados por rol: Auth/, Admin/, Company/, Student/.
app/Http/Middleware/	Middleware personalizados: RoleMiddleware (control de acceso por rol).
app/Notifications/	Clases de notificaciones por email y base de datos (3 notificaciones).
database/migrations/	Archivos que definen la estructura de la base de datos (tablas y columnas).
database/seeder/	Datos iniciales: admin, empresa y estudiante de prueba.
resources/views/	Plantillas Blade organizadas por módulo (admin/, company/, student/, auth/).
routes/web.php	Todas las rutas del sistema: públicas, de empresa, estudiante y admin.
public/	Lo que el navegador puede acceder directamente (index.php, assets compilados).
storage/	Archivos subidos por usuarios (CVs, logos) y logs del sistema.
config/	Configuraciones de Laravel: base de datos, correo, sesiones, servicios OAuth.
.env	Variables de entorno: credenciales de BD, correo, Google OAuth. NO se sube a GitHub.
composer.json	Lista de dependencias PHP del proyecto.
vite.config.js	Configuración de Vite para compilar los assets del frontend.
artisan	Herramienta CLI de Laravel. Ej: php artisan migrate.

Organización de Controladores por Módulo

Carpeta	Controlador	Responsabilidad
Auth/	CompanyAuthController	Login, registro de empresa y logout.
Auth/	GoogleController	Autenticación OAuth con Google para estudiantes.
Admin/	DashboardController	Panel admin: estadísticas, gestión de empresas y usuarios, reportes.
Company/	DashboardController	Panel empresa: perfil, postulaciones recibidas.
Company/	JobPostingController	CRUD de vacantes: crear, listar, editar, cerrar.
Student/	DashboardController	Panel estudiante: perfil, historial de postulaciones.
Student/	JobController	Listar vacantes disponibles, ver detalle, postularse.
/	HomeController	Página principal pública con vacantes destacadas.

5. Base de Datos — Modelos y Relaciones

La base de datos del sistema tiene 6 tablas principales más 3 tablas de infraestructura de Laravel (caché, colas y sesiones). A continuación se describe cada tabla, sus columnas más importantes y sus relaciones.

5.1 Tabla: users

Almacena todos los usuarios del sistema (admin, empresa y estudiante).

Columna	Tipo	Descripción
id	BIGINT PK	Identificador único autoincremental.
name	VARCHAR	Nombre completo del usuario.
email	VARCHAR	Correo electrónico único.
password	VARCHAR NULL	Contraseña hasheada (null para logins con Google).
role	ENUM	Rol del usuario: admin, student o company.
google_id	VARCHAR NULL	ID de Google cuando el usuario inicia sesión con OAuth.
avatar	VARCHAR NULL	URL de la foto de perfil (de Google o generada).
active	BOOLEAN	Indica si la cuenta está habilitada.
email_verified_at	TIMESTAMP	Fecha de verificación del correo.

5.2 Tabla: student_profiles

Extiende la información del usuario cuando es estudiante.

Columna	Tipo	Descripción
id	BIGINT PK	Identificador.
user_id	FK → users	Clave foránea. Cada perfil pertenece a un usuario.
student_code	VARCHAR	Código estudiantil de UNIPAZ.
program	VARCHAR	Programa académico (ej: Ingeniería de Sistemas).
semester	VARCHAR	Semestre actual.
phone	VARCHAR	Teléfono de contacto.
about	TEXT	Descripción personal / perfil profesional.
cv_path	VARCHAR	Ruta al archivo PDF de la hoja de vida.
linkedin	VARCHAR	URL del perfil de LinkedIn.

5.3 Tabla: companies

Almacena la información de las empresas registradas.

Columna	Tipo	Descripción
id	BIGINT PK	Identificador.
user_id	FK → users	Usuario asociado a esta empresa.
company_name	VARCHAR	Razón social de la empresa.
nit	VARCHAR	NIT de la empresa (único).
sector	VARCHAR	Sector económico (tecnología, salud, etc.).

Columna	Tipo	Descripción
contact_person	VARCHAR	Nombre del representante de la empresa.
status	ENUM	Estado: pending (espera), approved (aprobada), rejected (rechazada).
logo_path	VARCHAR	Ruta al logo de la empresa.
website	VARCHAR	Sitio web corporativo.

5.4 Tabla: job_postings

Cada fila es una vacante publicada por una empresa.

Columna	Tipo	Descripción
id	BIGINT PK	Identificador.
company_id	FK → companies	Empresa que publicó la vacante.
title	VARCHAR	Título del cargo (ej: Desarrollador Backend).
description	TEXT	Descripción completa del puesto.
requirements	TEXT	Requisitos para aplicar al cargo.
area	VARCHAR	Área de la empresa (Sistemas, Contabilidad, etc.).
contract_type	VARCHAR	Tipo de contrato (término fijo, indefinido, etc.).
modality	VARCHAR	Modalidad: onsite, remote o hybrid.
salary_range	VARCHAR	Rango salarial en texto (ej: "1.5M - 2M").
salary_negotiable	BOOLEAN	Si el salario es negociable.
positions	INTEGER	Número de vacantes disponibles.
deadline	DATE	Fecha límite para postularse.
status	ENUM	active (activa), paused (pausada), closed (cerrada).
requires_cv	BOOLEAN	Si la empresa requiere hoja de vida adjunta.

5.5 Tabla: applications

Registra cada postulación de un estudiante a una vacante.

Columna	Tipo	Descripción
id	BIGINT PK	Identificador.
user_id	FK → users	Estudiante que se postula.
job_posting_id	FK → job_postings	Vacante a la que se postula.
cover_letter	TEXT	Carta de presentación escrita por el estudiante.
cv_path	VARCHAR	CV adjunto específico para esta postulación.
status	ENUM	pending, reviewed, interview, accepted, rejected.
company_notes	TEXT	Notas internas de la empresa (no visibles al estudiante).
(UNIQUE)	user_id + job_id	Un estudiante no puede postularse dos veces a la misma vacante.

5.6 Relaciones entre tablas

Las relaciones permiten que Eloquent ORM recupere datos relacionados con una sola instrucción.

Tabla A	Relación	Tabla B	Descripción
users	1 → 1	student_profiles	Un usuario tiene un perfil de estudiante.

Tabla A	Relación	Tabla B	Descripción
users	1 → 1	companies	Un usuario puede ser dueño de una empresa.
users	1 → N	applications	Un usuario puede tener muchas postulaciones.
companies	1 → N	job_postings	Una empresa puede publicar muchas vacantes.
job_postings	1 → N	applications	Una vacante puede recibir muchas postulaciones.

6. Sistema de Autenticación

El sistema implementa dos mecanismos de autenticación según el tipo de usuario: login con Google para estudiantes y login tradicional con email/contraseña para empresas y administradores.

6.1 Autenticación de Estudiantes — Google OAuth 2.0

Los estudiantes inician sesión exclusivamente con su correo institucional @unipaz.edu.co a través de Google. Esto garantiza que solo miembros de la comunidad universitaria puedan acceder como estudiantes.

Pasos del flujo OAuth:

1. El estudiante hace clic en "Ingresar con Google".
2. Laravel Socialite redirige al servidor de autenticación de Google.
3. El estudiante inicia sesión en su cuenta de Google.
4. Google redirige de vuelta con un código de autorización.
5. Laravel intercambia el código por los datos del usuario (nombre, email, foto).
6. Se verifica que el email termine en @unipaz.edu.co.
7. Si es la primera vez, se crea el usuario con role="student" y su StudentProfile.
8. Si ya existe, se actualizan los datos de Google (avatar, google_id).
9. Se verifica que la cuenta esté activa (active = true).
10. Se hace login automático y redirige al dashboard del estudiante.

Código clave del GoogleController.php:

```
// Verificación de correo institucional
if (!str_ends_with($googleUser->getEmail(), '@unipaz.edu.co')) {
    return redirect()->route('login')
        ->with('error', 'Solo se permite el ingreso con correo @unipaz.edu.co');
}

// Buscar o crear el usuario
$user = User::where('google_id', $googleUser->getId())
    ->orWhere('email', $googleUser->getEmail())->first();
if (!$user) {
    $user = User::create([
        'name' => $googleUser->getName(), 'email' => $googleUser->getEmail(),
        'google_id' => $googleUser->getId(), 'role' => 'student',
    ]);
    StudentProfile::create(['user_id' => $user->id, 'program' => 'Sin especificar']);
}
```

6.2 Autenticación de Empresas y Admin — Email y Contraseña

Las empresas y el administrador ingresan con email y contraseña. La contraseña se almacena con bcrypt (hashing seguro, nunca en texto plano). Después del login, el sistema redirecciona al dashboard correspondiente según el rol.

```
if (Auth::attempt($credentials, $request->boolean('remember'))) {
    $user = Auth::user();
    if (!$user->active) { Auth::logout(); return back()->with('error', '...'); }
    return match ($user->role) {
        'admin' => redirect()->route('admin.dashboard'),
        'company' => redirect()->route('company.dashboard'),
        'student' => redirect()->route('student.dashboard'),
    };
}
```

6.3 Registro de Empresas

Las empresas se registran a través de un formulario público. Al registrarse se crean dos registros: uno en la tabla users y otro en la tabla companies. La empresa queda con status = "pending" hasta que el administrador la apruebe.

6.4 Middleware de Roles — RoleMiddleware

El sistema protege cada ruta verificando que el usuario autenticado tenga el rol correcto. El middleware se encadena en las rutas de esta forma:

```
// Solo el admin puede acceder
Route::middleware(['auth', 'role:admin'])->group(function () { ... });

// Solo las empresas
Route::middleware(['auth', 'role:company'])->group(function () { ... });

// Solo los estudiantes
Route::middleware(['auth', 'role:student'])->group(function () { ... });
```

7. Módulo de Administración

El administrador de UNIPAZ tiene control total sobre el sistema. Su panel es accesible en /admin/dashboard y solo los usuarios con role = "admin" pueden ingresar.

Funcionalidades del panel de administración

Funcionalidad	Ruta	Descripción
Dashboard principal	/admin/dashboard	Estadísticas globales: total de estudiantes, empresas, vacantes, postulaciones y gráficos.
Gestión de empresas	/admin/empresas	Lista todas las empresas. Permite aprobar o rechazar las que están en estado "pending".
Gestión de usuarios	/admin/usuarios	Lista todos los usuarios. El admin puede activar o desactivar cualquier cuenta.
Reportes	/admin/reportes	Estadísticas detalladas: empresas por estado, postulaciones por estado, vacantes por área.

¿Cómo aprueba el admin una empresa?

Cuando una empresa se registra, su status queda en "pending". El administrador ingresa al panel, hace clic en "Aprobar" y el sistema ejecuta este código:

```
public function approveCompany(Company $company)
{
    $company->update(['status' => 'approved']);
    // Envía notificación automática al empresario por email
    $company->user->notify(new CompanyApprovedNotification($company));
    return back()->with('success', 'Empresa aprobada.');
```

Al aprobar la empresa, se dispara automáticamente la notificación CompanyApprovedNotification que envía un correo al empresario informándole que ya puede publicar vacantes.

8. Módulo de Empresas

Una vez que la empresa es aprobada por el administrador, puede publicar vacantes y gestionar las postulaciones. Su panel se encuentra en `/empresa/dashboard`.

Funcionalidades del panel de empresa

Funcionalidad	Ruta	Descripción
Dashboard	<code>/empresa/dashboard</code>	Resumen: número de vacantes activas, total de postulaciones, últimas postulaciones recibidas.
Perfil	<code>/empresa/perfil</code>	Ver y actualizar información de la empresa (nombre, sector, NIT, logo, descripción, etc.).
Mis Vacantes	<code>/empresa/vacantes</code>	Lista de todas sus vacantes con número de postulaciones. Crear, editar o cerrar.
Crear Vacante	<code>/empresa/vacantes/create</code>	Formulario para publicar una nueva vacante con todos sus atributos.
Postulaciones	<code>/empresa/postulaciones</code>	Ver los estudiantes que se postularon. Puede cambiar el estado y agregar notas.

CRUD de Vacantes — JobPostingController

El controlador `JobPostingController` implementa las 5 operaciones del CRUD usando el Resource Controller de Laravel:

Método HTTP	Ruta	Método	Acción
GET	<code>/empresa/vacantes</code>	<code>index()</code>	Lista todas las vacantes de la empresa.
GET	<code>/empresa/vacantes/create</code>	<code>create()</code>	Muestra el formulario de nueva vacante.
POST	<code>/empresa/vacantes</code>	<code>store()</code>	Valida y guarda la nueva vacante en BD.
GET	<code>/empresa/vacantes/{id}/edit</code>	<code>edit()</code>	Muestra el formulario de edición.
PUT	<code>/empresa/vacantes/{id}</code>	<code>update()</code>	Valida y actualiza la vacante en BD.
DELETE	<code>/empresa/vacantes/{id}</code>	<code>destroy()</code>	Cambia el status a "closed".

Validación al crear una vacante (extracto)

```
$request->validate([
    'title' => 'required|string|max:200',
    'description' => 'required|string|max:5000',
    'requirements' => 'required|string|max:3000',
    'modality' => 'required|in:onsite,remote,hybrid',
    'deadline' => 'required|date|after:today',
    'positions' => 'required|integer|min:1|max:100',
]);
```

Estados de las postulaciones

Estado	Significado	Color en la interfaz
pending	Recién recibida, aún sin revisar.	Gris (secondary)
reviewed	La empresa ya vio la postulación.	Azul (info)
interview	El estudiante fue citado a entrevista.	Amarillo (warning)
accepted	El estudiante fue seleccionado.	Verde (success)
rejected	El estudiante no fue seleccionado.	Rojo (danger)

9. Módulo de Estudiantes

El estudiante es el actor central del sistema. Su experiencia comienza con el login mediante Google y continúa con la búsqueda y postulación a vacantes.

Funcionalidades del panel de estudiante

Funcionalidad	Ruta	Descripción
Dashboard	/estudiante/dashboard	Resumen personal: postulaciones activas, vacantes disponibles, estado de perfil.
Mi Perfil	/estudiante/perfil	Datos: código estudiantil, programa, semestre, teléfono, descripción, LinkedIn, subir CV.
Buscar Vacantes	/estudiante/vacantes	Lista de vacantes activas con filtro por área. Paginación de 12 vacantes por página.
Ver Vacante	/estudiante/vacantes/{id}	Detalle completo: descripción, empresa, requisitos, tipo de contrato, salario.
Postularse	POST /vacantes/{id}/postular	Formulario de postulación: carta de presentación y CV adjunto opcional.
Mis Postulaciones	/estudiante/postulaciones	Historial de todas las postulaciones con su estado actual.

¿Cómo se postula un estudiante?

La lógica de postulación está en el método `apply()` del `Student/JobController.php`:

```
public function apply(Request $request, JobPosting $jobPosting)
{
    if (!$jobPosting->isActive()) {
        return back()->with('error', 'Esta vacante ya no está disponible.');
```

```
    }
    // Verificar que no se haya postulado antes
    $existing = Application::where('user_id', auth()->id())
        ->where('job_posting_id', $jobPosting->id)->first();
    if ($existing) { return back()->with('error', 'Ya te postulaste.');
```

```
    }
    // Guardar la postulación
    $application = Application::create([
        'user_id' => auth()->id(), 'job_posting_id' => $jobPosting->id,
        'cover_letter' => $request->cover_letter, 'status' => 'pending',
    ]);
    // Notificar a la empresa
    $jobPosting->company->user->notify(new NewApplicationNotification($application));
    return back()->with('success', 'Postulacion enviada exitosamente.');
```

```
}
```

Query Scopes en el modelo JobPosting

El modelo `JobPosting` tiene Query Scopes que permiten reutilizar filtros comunes de manera elegante:

```
// Scope: solo vacantes activas y vigentes
public function scopeActive($query)
{
    return $query->where('status', 'active')->where('deadline', '>=', now());
}
```

```
// Scope: filtrar por área
public function scopeByArea($query, $area)
{
    return $query->where('area', $area);
}
```

```
}  
  
// Uso en el controlador:  
$vacantes = JobPosting::active()->byArea('Sistemas')->paginate(12);
```

10. Sistema de Notificaciones

El sistema cuenta con 3 clases de notificaciones que se envían automáticamente en momentos clave del proceso. Cada notificación usa dos canales: email (correo electrónico) y database (almacenada en BD para mostrar en el panel del usuario).

Clase	Cuándo se dispara	Destinatario
CompanyApprovedNotification	El admin aprueba una empresa.	El usuario dueño de la empresa.
NewApplicationNotification	Un estudiante se postula a una vacante.	El usuario dueño de la empresa con esa vacante.
ApplicationStatusNotification	La empresa cambia el estado de una postulación.	El estudiante que se postuló.

¿Cómo funcionan las notificaciones en Laravel?

Las notificaciones implementan la interfaz ShouldQueue, lo que significa que se procesan de forma asíncrona en segundo plano. El proceso es:

- 1. Se llama a `$user->notify(new MiNotificacion($datos))`.
- 2. Laravel encola la tarea en la tabla "jobs" de la base de datos.
- 3. El worker (php artisan queue:work) procesa la cola en segundo plano.
- 4. Se envía el correo electrónico al destinatario.
- 5. Se guarda un registro en la tabla "notifications" para mostrarlo en el panel.

Estructura de una Notificación

```
class NewApplicationNotification extends Notification implements ShouldQueue
{
    use Queueable;
    public function __construct(public Application $application) {}

    // Define los canales: email + base de datos
    public function via(object $notifiable): array
    { return ['mail', 'database']; }

    // Contenido del email
    public function toMail(object $notifiable): MailMessage { ... }

    // Datos que se guardan en la tabla notifications
    public function toArray(object $notifiable): array
    { return ['tipo' => 'nueva_postulacion', 'mensaje' => '...']; }
}
```

11. Rutas y Middleware

El archivo `routes/web.php` organiza todas las URLs del sistema. Las rutas están agrupadas por rol, lo que simplifica la aplicación de middleware y hace el código más legible.

Grupos de rutas y sus protecciones

Prefijo	Middleware	Descripción
/ (raíz)	ninguno	Página pública: lista vacantes destacadas.
/login, /register	guest	Solo para usuarios no autenticados. Si ya está logueado, redirige al dashboard.
/auth/google	guest	Flujo OAuth de Google para estudiantes.
/logout	auth	Cierra la sesión del usuario actual.
/admin/*	auth + role:admin	Solo el administrador puede acceder.
/empresa/*	auth + role:company	Solo empresas autenticadas.
/estudiante/*	auth + role:student	Solo estudiantes autenticados.

Named Routes (Rutas con nombre)

Todas las rutas tienen un nombre. Esto permite generar URLs en las vistas y controladores sin hardcodear la URL, facilitando cambios futuros:

```
// En una vista Blade:
<a href="{{ route('student.jobs') }}">Ver vacantes</a>

// En un controlador:
return redirect()->route('company.dashboard');
```

Middleware CheckRole — Cómo funciona

```
public function handle(Request $request, Closure $next, string ...$roles): Response
{
    // 1. Verificar que el usuario esté autenticado
    if (!Auth::check()) { return redirect()->route('login'); }

    // 2. Verificar que la cuenta esté activa
    if (!$user->active) {
        Auth::logout();
        return redirect()->route('login')->with('error', 'Cuenta desactivada.');
```

Nota: El parámetro `...$roles` permite pasar uno o varios roles al middleware. Por ejemplo: `->middleware("role:admin,company")` permitiría acceder tanto a administradores como a empresas.

12. Vistas — Blade Templates

Blade es el motor de plantillas de Laravel. Los archivos Blade tienen extensión `.blade.php` y permiten combinar HTML con lógica PHP de forma limpia y legible.

Sintaxis básica de Blade

Sintaxis Blade	Equivalente PHP	Uso
<code>{{ \$variable }}</code>	<code><?= htmlspecialchars(\$var) ?></code>	Mostrar variable con protección XSS.
<code>{!! \$html !!}</code>	<code><?= \$html ?></code>	Mostrar HTML sin escapar (con cuidado).
<code>@if / @else / @endif</code>	<code>if / else</code>	Condicionales.
<code>@foreach / @endforeach</code>	<code>foreach</code>	Iterar sobre colecciones.
<code>@extends("layout")</code>	<code>--</code>	Heredar de una plantilla base.
<code>@section("nombre")</code>	<code>--</code>	Definir una sección de contenido.
<code>@yield("nombre")</code>	<code>--</code>	Insertar el contenido de una sección.
<code>@include("partial")</code>	<code>--</code>	Incluir un subcomponente Blade.
<code>@auth / @guest</code>	<code>if Auth::check()</code>	Condicional según si el usuario está logueado.

Organización de las vistas

Carpeta	Vistas principales	Función
layouts/	app.blade.php, admin.blade.php	Plantillas base que definen la estructura HTML compartida.
auth/	login.blade.php, company-register.blade.php	Formularios de login y registro de empresa.
admin/	dashboard, companies/, users/, reports	Panel del administrador.
company/	dashboard, jobs/, applications	Panel de la empresa.
student/	dashboard, jobs/, applications	Panel del estudiante.
home.blade.php	--	Página pública principal con vacantes destacadas.
errors/	403.blade.php, 404.blade.php	Páginas de error personalizadas.

Ejemplo de vista Blade — Lista de vacantes:

```
@extends('layouts.app')
@section('content')
    <h2>Vacantes disponibles</h2>
    @foreach($jobPostings as $job)
        <div class='card mb-3'>
            <h5>{{ $job->title }}</h5>
            <p>{{ $job->company->company_name }}</p>
            <a href="{{ route('student.jobs.show', $job) }}">Ver detalle</a>
        </div>
    @endforeach
    {{-- Paginación --}}
    {{ $jobPostings->links() }}
@endsection
```

13. Instalación Local — Paso a Paso

Sigue estos pasos para ejecutar el proyecto en tu máquina local. Necesitas tener instalados: PHP 8.2+, Composer, Node.js 18+ y Git.

Paso 1: Descomprimir el proyecto

Descomprime el archivo ZIP del proyecto en una carpeta de tu computadora.

Paso 2: Instalar dependencias PHP

Abre una terminal en la carpeta del proyecto y ejecuta:

```
composer install
```

Paso 3: Configurar el archivo .env

Copia el archivo de configuración de ejemplo y genera la clave de la aplicación:

```
cp .env.example .env
php artisan key:generate
```

Paso 4: Ejecutar migraciones y datos de prueba

Esto crea todas las tablas en la base de datos SQLite y carga los datos iniciales:

```
php artisan migrate
php artisan db:seed
```

Paso 5: Configurar el almacenamiento

Crea el enlace simbólico para que los archivos subidos sean accesibles:

```
php artisan storage:link
```

Paso 6: Iniciar el servidor de desarrollo

```
php artisan serve
```

Abre tu navegador en: <http://localhost:8000>

Credenciales de acceso para pruebas

Rol	Correo	Contraseña
Administrador	admin@unipaz.edu.co	Admin2024*
Empresa	info@tecnosoluciones.com	Empresa2024*
Estudiante	Login con Google @unipaz.edu.co	—

Configurar Google OAuth (para login de estudiantes)

- 1. Ve a <https://console.cloud.google.com> y crea un proyecto nuevo.
- 2. En APIs y servicios → Credenciales → Crear credencial → ID de cliente OAuth.
- 3. Tipo: Aplicación web. URI de redireccionamiento: <http://localhost:8000/auth/google/callback>
- 4. Copia el Client ID y Client Secret al archivo .env:

```
GOOGLE_CLIENT_ID=tu-client-id.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=tu-client-secret
GOOGLE_REDIRECT_URI=http://localhost:8000/auth/google/callback
```

Configurar envío de correos (para notificaciones)

```
MAIL_MAILER=smt
MAIL_HOST=smt
MAIL_PORT=587
```

```
MAIL_USERNAME=tu-correo@gmail.com
MAIL_PASSWORD=xxxx-xxxx-xxxx-xxxx # App Password de Google
MAIL_FROM_ADDRESS=bolsaempleo@unipaz.edu.co
```

Iniciar el worker de colas

```
# En otra terminal, mientras php artisan serve esta corriendo:
php artisan queue:work
```

14. Despliegue en Railway (Producción)

Railway.app es una plataforma de cloud hosting que permite desplegar aplicaciones PHP/Laravel de forma gratuita conectando el repositorio de GitHub. El proyecto ya incluye los archivos `start.sh` y `railway.json` configurados.

Paso 1: Subir el código a GitHub

```
git init
git add .
git commit -m "Deploy inicial"
git remote add origin https://github.com/TU_USUARIO/bolsa-empleo-unipaz.git
git push -u origin main
```

Paso 2: Crear proyecto en Railway

Ingresa a railway.app con tu cuenta de GitHub. Haz clic en "New Project" → "Deploy from GitHub" y selecciona tu repositorio.

Paso 3: Configurar variables de entorno

En Railway, ve a la pestaña Variables y agrega estas configuraciones:

Variable	Valor recomendado
APP_NAME	Bolsa de Empleo UNIPAZ
APP_ENV	production
APP_DEBUG	false
DB_CONNECTION	sqlite
SESSION_DRIVER	file
QUEUE_CONNECTION	database
GOOGLE_CLIENT_ID	(tu ID de Google OAuth)
GOOGLE_CLIENT_SECRET	(tu secret de Google OAuth)
APP_URL	https://tu-app.up.railway.app
GOOGLE_REDIRECT_URI	https://tu-app.up.railway.app/auth/google/callback

Paso 4: Generar el dominio

En Settings → Networking → "Generate Domain". Copia la URL y agrégala a las variables APP_URL y GOOGLE_REDIRECT_URI.

Paso 5: Actualizar Google Cloud Console

Agrega la nueva URL de Railway como URI autorizado en tu cliente OAuth de Google.

Actualizar el proyecto en producción

Cada vez que hagas cambios locales, Railway redespigará automáticamente al hacer push:

```
git add .
git commit -m "descripcion del cambio"
git push
# Railway detecta el push y ejecuta el redeploy automaticamente
```

15. Glosario de Términos

Definición de los términos técnicos más usados en este proyecto y en el desarrollo web con Laravel.

Término	Definición
Artisan	Herramienta de línea de comandos de Laravel. Permite ejecutar migraciones, seeders, crear archivos, etc. Uso: <code>php artisan [comando]</code> .
Blade	Motor de plantillas HTML de Laravel. Los archivos <code>.blade.php</code> mezclan HTML con instrucciones de Laravel como <code>@foreach</code> , <code>@if</code> , <code>{{ \$variable }}</code> .
CRUD	Create, Read, Update, Delete. Las 4 operaciones básicas de cualquier sistema sobre una base de datos.
Eloquent ORM	El ORM de Laravel. Permite interactuar con la base de datos usando objetos PHP en lugar de escribir SQL directamente.
Facade	Clase estática que provee acceso simplificado a servicios de Laravel. Ej: <code>Auth::user()</code> , <code>Route::get()</code> , <code>Cache::put()</code> .
FK (Foreign Key)	Clave foránea. Una columna que referencia el ID de otra tabla para crear una relación entre ellas.
Middleware	Capa de código que se ejecuta antes de que una petición llegue al controlador. Ej: verificar si el usuario está autenticado.
Migration	Archivo PHP que define la estructura de una tabla de la base de datos. Son versionados con Git.
MVC	Modelo-Vista-Controlador. Patrón que separa los datos (modelo), la presentación (vista) y la lógica de control.
OAuth 2.0	Protocolo de autorización que permite a una aplicación acceder a recursos de otra (ej: Google) con el permiso del usuario.
ORM	Object-Relational Mapping. Técnica que convierte filas de una base de datos en objetos PHP.
Paginate	Dividir un conjunto de resultados en páginas. En Laravel: <code>JobPosting::all()->paginate(12)</code> muestra 12 por página.
Queue (Cola)	Sistema para ejecutar tareas en segundo plano sin bloquear la respuesta al usuario. Usado para enviar correos.
Route	Definición de una URL y el controlador que la maneja. Definidas en <code>routes/web.php</code> .
Seeder	Archivo PHP que inserta datos iniciales en la base de datos. Útil para datos de prueba.
Scope (Query)	Método reutilizable en un modelo que encapsula un filtro de base de datos. Ej: <code>JobPosting::active()</code> .
SQLite	Motor de base de datos que guarda todos los datos en un solo archivo <code>.sqlite</code> . Ideal para desarrollo y demos.
Tinker	REPL de Laravel que permite ejecutar código PHP interactivo para probar modelos y consultas.
Validation	Proceso de verificar que los datos de un formulario cumplan ciertas reglas antes de guardarlos en la BD.

Deploy en Railway — Bolsa de Empleo UNIPAZ

EXPLORADOR

BOLSA-EMPLEO-UNIPAZ-BASE

- app
 - Http
 - Controllers
 - Middleware
 - CompanyApprovedMiddleware.php
 - RoleMiddleware.php
 - Requests
 - Kernel.php
 - Models
 - Notifications
 - Providers
- bootstrap
- cache
 - app.php
 - providers.php
- config
- database
- public
- resources
 - css
 - app.css
 - js
- views

ESQUEMA

LÍNEA DE TIEMPO

MASCOTAS DE VS CODE

startsh

.env.example

DEPLOY-RAILWAY.md

BLACKBOX

DEPLOY-RAILWAY.md > abc # Deploy en Railway — Bolsa de Empleo UNIPAZ > abc ## Paso 5 — Actualizar Google OAuth

```
1 # Deploy en Railway – Bolsa de Empleo UNIPAZ
2
3 ## Requisitos previos
4 - Cuenta en [github.com](https://github.com) (gratis)
5 - Cuenta en [railway.app](https://railway.app) (gratis con GitHub)
6
7 ---
8
9 ## Paso 1 – Subir el proyecto a GitHub
10
11 Abre una terminal en la carpeta del proyecto y ejecuta:
12
13 ```bash
14 git init
15 git add .
16 git commit -m "Bolsa de Empleo UNIPAZ - deploy inicial"
17 ```
18
```

PROBLEMAS 18 SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS DEVDB

PS E:\xampp\htdocs\Laravel\bolsa-empleo-unipaz-base> php artisan serve

2026-04-20 21:11:40	/images/Logowhite.png	~ 0.22ms
2026-04-20 21:11:40	/images/LOGO-UNIPAZ-200.png	~ 0.91ms
2026-04-20 21:11:40	/images/Unipaz-Avanza.png	~ 1.59ms
2026-04-20 21:11:41	/favicon.ico	~ 0.51ms
2026-04-21 07:43:37	/login	~ 510.93ms
2026-04-21 07:43:38	/images/Logowhite.png	~ 0.09ms
2026-04-21 07:43:38	/images/LOGO-UNIPAZ-200.png	~ 0.85ms
2026-04-21 07:43:38	/images/Unipaz-Avanza.png	~ 0.11ms
2026-04-21 07:43:38	/favicon.ico	~ 0.24ms

main 17 1 BLACKBOX Agent Open Website Generate Commit Message Go Live BLACKBOXAI: Open Chat Windsurf (Pre-Release): Prettier